Exercício Extra: Sistema BLE

Tecnologias de Comunicação para a Internet das Coisas, Aula 06

Catarina Silva

ESTGA, Universidade de Aveiro | 2026

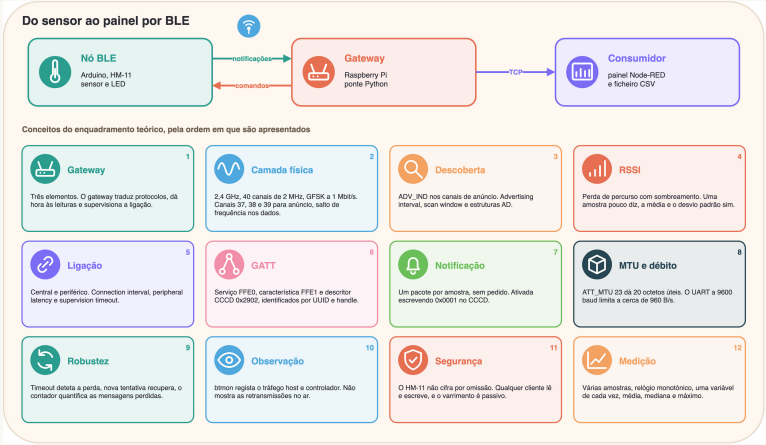
Agenda

- O que vão construir e o que vão demonstrar
- O enquadramento do sistema, num resumo gráfico
- As fases do exercício e o que fazer em cada uma
- O código disponível, o que é novo e como o adaptar
- Resultados para demonstração e regras da apresentação

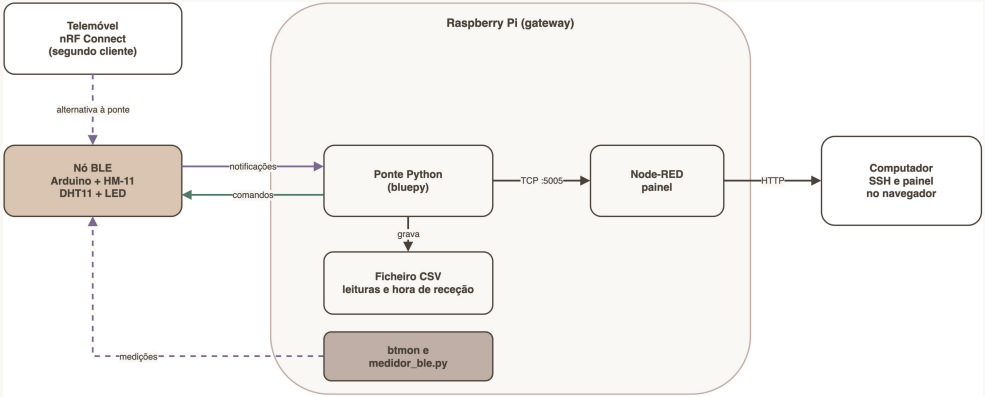
O Que Vão Construir

- Um posto de monitorização de uma sala: um nó com Arduino e módulo HM-11 mede e é comandado por BLE.
- Um Raspberry Pi liga-se ao nó, recebe as leituras, regista-as e apresenta-as num painel.
- O foco não está no painel, mas no BLE: no fim, cada grupo demonstra e explica resultados que o sustentem.
- Cabe numa aula: basta a primeira fase a funcionar, e a parte de código mais pesada é fornecida.

Resumo Gráfico

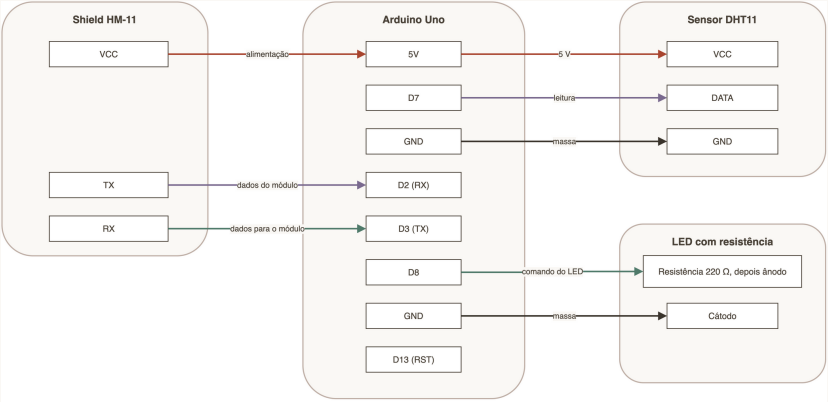


Arquitetura do Sistema



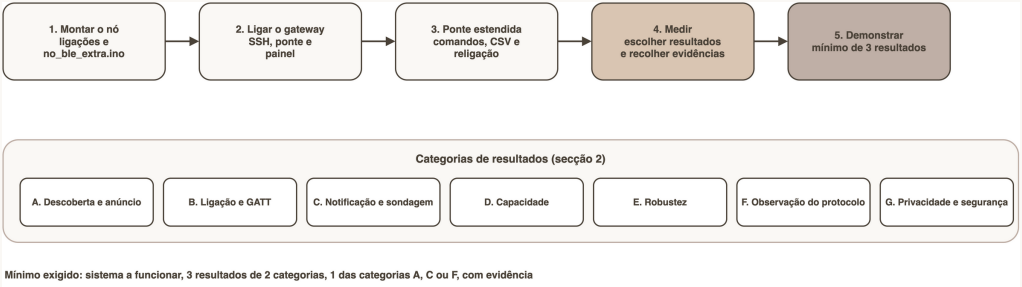
O HM-11 aceita um cliente BLE de cada vez: a ponte, o medidor e o telemóvel não podem estar ligados em simultâneo.

Ligações no Arduino



O pino D13 comanda o reset do módulo no arranque (sem fio externo). Os dois pinos GND são equivalentes.

Etapas do Exercício



Mínimo exigido: sistema a funcionar, 3 resultados de 2 categorias, 1 das categorias A, C ou F, com evidência

Fases de Realização

- Fase 1: nó, ponte da Aula 05 e painel com um gráfico. É o mínimo para demonstrar.
- Fase 2: ponte estendida, com comandos ao nó e ficheiro CSV com a hora de receção.
- Fase 3: religação automática testada e indicador do estado da ligação no painel.
- Demonstração: o sistema a funcionar e pelo menos 3 resultados, de 2 categorias, com 1 das categorias A, C ou F.
- O DHT11 e o LED são opcionais: sem eles o exercício faz-se igual.

Código Disponível

Ficheiro	Origem	Para que serve	Como usar
ponte_ble_tcp.py	Aula 05	Liga ao nó, ativa as notificações e reenvia-as por TCP (porta 5005).	sudo python3 ponte_ble_tcp.py <ENDEREÇO>
varredor.py	Novo	Lista os dispositivos BLE próximos com endereço e RSSI.	sudo python3 varredor.py 15
ponte_estendida.py	Novo, da ponte da Aula 05	Comandos, CSV com hora, religação e contagem de perdas.	sudo python3 ponte_estendida.py <ENDEREÇO> leituras.csv
no_ble_extra.ino	Novo, do servidor da Aula 05	Nó com LED, envio espontâneo, contador e modo simulado.	Carregar no Arduino.
ponte_at.ino	Novo	Configura o HM-11 por comandos AT.	Monitor Série a 9600 baud.
medidor_ble.py	Novo	Mede RSSI, latência e cadência e grava em CSV.	sudo python3 medidor_ble.py <modo> <ENDEREÇO>

O Que Já Usaram e o Que é Novo

- Já usado: a ponte `ponte_ble_tcp.py`, o painel Node-RED com `tcp in`, `function` e `ui-gauge`, e os comandos `t`, `h` e `X` do servidor da Aula 05.
- Novo no nó: LED (`1`, `0`), envio espontâneo (`S`, `P`), período (`a` a `e`), contador nas mensagens e modo sem sensores.
- Novo no Raspberry Pi: o varredor, o medidor, a configuração do módulo por AT e a ponte estendida.
- Para adaptar: a ponte estendida já traz os comandos, o CSV e a religação. O trabalho é executá-la, lê-la e alterá-la.

Como Adaptar o Código (1/3)

- `no_ble_extra.ino` substitui o servidor da Aula 05. Os comandos `t`, `h` e `X` mantêm-se.
- Com `X`, a mensagem passa a ser `contador,temperatura,humidade`. No painel, o nó `function` passa a usar as posições 1 e 2 em vez de 0 e 1.
- A ponte da Aula 05 envia apenas o comando `X` em ciclo. É o ponto de partida para os requisitos do exercício.

Como Adaptar o Código (2/3)

- Executar a ponte estendida: cada linha escrita no terminal é enviada ao nó como comando, por exemplo 1 para acender o LED.
- O ficheiro CSV regista a hora de cada mensagem e de cada evento de ligação, com as mensagens perdidas contadas a partir do contador do nó.
- Se a ligação cair, a ponte volta a tentar com espera crescente, ativa de novo as notificações e retoma o envio.

Como Adaptar o Código (3/3)

- A opção `-sondagem` usa pedidos periódicos em vez do envio espontâneo, para comparar os dois modos.
- Adaptações possíveis: período de envio, formato das mensagens, campos do CSV, outro sensor.
- A ponte, o medidor e o telemóvel não podem estar ligados ao nó em simultâneo.
- Depois de configurar por AT, voltar a carregar `no_ble_extra.ino`.

Comandos do Nó

Caracter	Efeito
`t, h, X`	Uma leitura: temperatura, humidade, ou `contador,temperatura,humidade`.
`1, 0`	Acende e apaga o LED.
`S, P`	Inicia e para o envio espontâneo.
`a, b, c, d, e`	Período do envio espontâneo: 1, 2, 5, 10 e 30 segundos.

Todos os comandos são escritos na característica `FFE1`.

Usar o medidor_ble.py

- Modo `rssi`: varre repetidamente e regista o RSSI do endereço indicado. Serve os resultados A3, A4, A5 e A6.
- Modo `latencia`: escreve o comando `X` e mede o tempo até chegar a notificação. Serve o resultado C2.
- Modo `cadencia`: regista o instante de cada notificação, com o envio espontâneo ativo. Serve o resultado C3.
- Forma geral: `sudo python3 medidor_ble.py <modo> <ENDEREÇO> [n] [saida.csv]`.

Painel Node-RED

- Reutilizar o painel da Aula 05: `tcp in` em `127.0.0.1:5005`, com *strings* delimitadas por fim de linha, ligado a um nó `function`.
- Alterar o conteúdo do nó `function`, que passa a ter duas saídas com as posições 1 e 2:

```
var v = msg.payload.split(",");
return [ { payload:  parseFloat(v[1]) },
         { payload:  parseFloat(v[2]) } ];
```

- Acrescentar um indicador do estado da ligação, que assinala a ausência de leituras durante um certo intervalo.

Acesso ao Raspberry Pi

- O Raspberry Pi funciona sem monitor nem teclado, como um gateway real: o trabalho faz-se por SSH.

```
ssh pi@<ENDEREÇO_DO_RASPBERRY>
hciconfig
sudo hciconfig hci0 up
sudo python3 varredor.py 15
```

- Os dois primeiros comandos após o SSH confirmam que o adaptador existe e ativam-no. O varredor devolve o endereço do nó.

Sem Todos os Sensores

- O exercício precisa do HM-11 ligado ao Arduino e de mais nada: os resultados dizem respeito ao BLE, não ao sensor.
- No início de `no_ble_extra.ino`, `USAR_SENSOR` a 0 simula as leituras, sem DHT11 nem biblioteca.
- `USAR_LED` a 0 dispensa o LED: o comando `1/0` é confirmado no Monitor Série. O pino 13 não serve de LED, porque comanda o reset do HM-11.
- Com outros sensores, a alteração faz-se na função `enviar()`. Exemplos: DHT22, LDR, sensor ultrassónico, PIR ou botão.
- Os pinos D2 e D3 estão ocupados pelo módulo e o D13 pelo seu reset.

Se Faltar Material

- Cada grupo precisa de um nó, com Arduino e HM-11, e de um Raspberry Pi.
- Se faltar um deles, dois grupos partilham o material e alternam, porque o HM-11 aceita um cliente de cada vez.
- Enquanto um grupo mede, o outro prepara o painel, lê o código ou organiza as evidências.

Resultados para Demonstração

- Escolhem-se resultados de sete categorias, cada um com a sua evidência: um número, um gráfico ou uma captura.
- A, descoberta e anúncio: nome, dados de anúncio, RSSI e distância, advertising interval, potência de emissão.
- B, ligação e GATT: perfil, CCCD, escrita com e sem confirmação.
- C, notificação e sondagem: tráfego, latência, cadência, comando ao LED.
- D, capacidade: fragmentação, débito, peso do cabeçalho.
- E, robustez: tempo até perda, religação, ligação exclusiva, interferência.
- F, observação do protocolo: sequência no `btmon`, notificação em hexadecimal, parâmetros da ligação.
- G, privacidade e segurança: dispositivos visíveis, acesso sem emparelhamento.

Demonstração

- Mostrar o sistema a funcionar, pelo menos na fase 1.
- Escolher pelo menos 3 resultados, de pelo menos 2 categorias, dos quais pelo menos 1 das categorias A, C ou F.
- O nível de cada resultado indica o esforço: 1 é uma observação direta, 2 uma medição repetida, 3 uma medição, captura ou modificação ao sistema.
- Cada resultado tem de estar pronto a mostrar, sem novas medições. O enunciado tem as tabelas completas com a evidência esperada.

Síntese

- O exercício reúne nó, gateway e painel num só sistema, usado para observar e medir o BLE.
- Basta a fase 1 a funcionar. A ponte estendida e o modo sem sensores estão fornecidos, para caber na aula.
- Os ficheiros novos servem o nó, a configuração, as medições e a ponte estendida.
- Cada resultado apresentado tem de ter evidência e uma explicação ligada ao protocolo.

Fontes

Gonçalves, P. *guia 4 — arduinoBLE*, TCIoT, edições anteriores. Base da ponte e do servidor BLE.

Documentação da biblioteca bluepy: `ianharvey.github.io/bluepy-doc/`

Documentação do BlueZ e da ferramenta `btmon`: `bluez.org`

Folha de dados do módulo HM-11, com a lista de comandos AT.

Bom trabalho.

Questões e comentários